Of the three different ratios, you should stay away from either extreme (the examples on the left and right in figure 10.10). In the first example, a small intersection between assignees and creators typically can lead to members of the team who specialize in creating requirements and filling out tasks and other team members who just work the specs they're given. You'll see symptoms of this when you start to hear people complaining about requirements or blaming requirements for poor or incorrect implementations.

In the example on the right in figure 10.10, on teams where developers are responsible for creating their own tasks, you can start to see people entering the bare minimum in the system to indicate they have a task to work on. This extreme also tends to lead to poor and incomplete requirements, and this becomes a problem when people forget what their minimal cards meant and try to estimate them, or if work ends up getting divided among other team members.

The second example (the one in the middle) is usually a good mix of members inside the team and outside stakeholders entering tasks into the work stream. This mix typically gives the development team enough investment in their work but also has enough outside influence to keep requirements complete.

An effective team creates effective software using effective processes. The final element is the requirements these teams use to build their software. The last agile principle belongs in a category by itself; although it's only a single line, it speaks to how your team defines what they're doing before they do it and can be measured from a few dimensions.

10.5. One principle for effective requirements

The final question you need to answer is "Are your requirements effective?"

- Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.